CIS 4910

Week 7: Patterns, AI, and Methods

Design patterns in depth, a real working relationship with AI in detailed design, and the methodology that carries your team through the build.

Professor Scot Hollingsworth

scoth@usf.edu

Office hours by appointment only



Agenda for Week 7

01 Design Patterns, Deep Dive

02 AI in Design and Architecture

03 AI Pitfalls and Guardrails

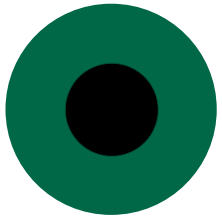
04 Methodology: Agile, AI SE, TDD, FDD

05 Takeaways

BY THE END OF TODAY

Learning Objectives

- Recognize and correctly apply patterns from all three GoF families (Gang of 4: 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software)*
- Choose a pattern using a repeatable decision process, not a favorite
- Write a prompt that gets AI to produce a usable design proposal
- Write an ADR that records an AI assisted decision the team can trust
- Recognize six specific AI pitfalls before they cost you a sprint





01

Design Patterns, Deep Dive

Every family, and how to choose between them

Factory Method, Repository, and Observer

Factory Method

Creates a Ticket, EscalatedTicket, or BugTicket without the caller knowing which in advance. Use when new related types should not touch existing code. Covered in Stop 2, Creating the Work.

Repository

Gives the rest of the application one clean, collection like path to the data store. Use when you want to isolate data access so storage can change later. Covered in Stop 3, One Path to Your Data.

Observer

Notifies every interested part of the system when a Ticket's state changes. Use when several parts must react to one event. Covered in Stop 5, Reacting and Recording.

What Makes a Pattern AI Centric

- The classic patterns did not change. Factory Method, Repository, and Observer solve the same problems they always did.
- What changed is who writes the code and how fast it changes.
- An AI centric pattern is a classic pattern applied at a new seam: where your system calls an AI model, or where it grounds that model in your own data.
- Model Gateway and Retrieval Augmented Generation, RAG, exist because of that seam. Every other pattern here is simply viewed through it.

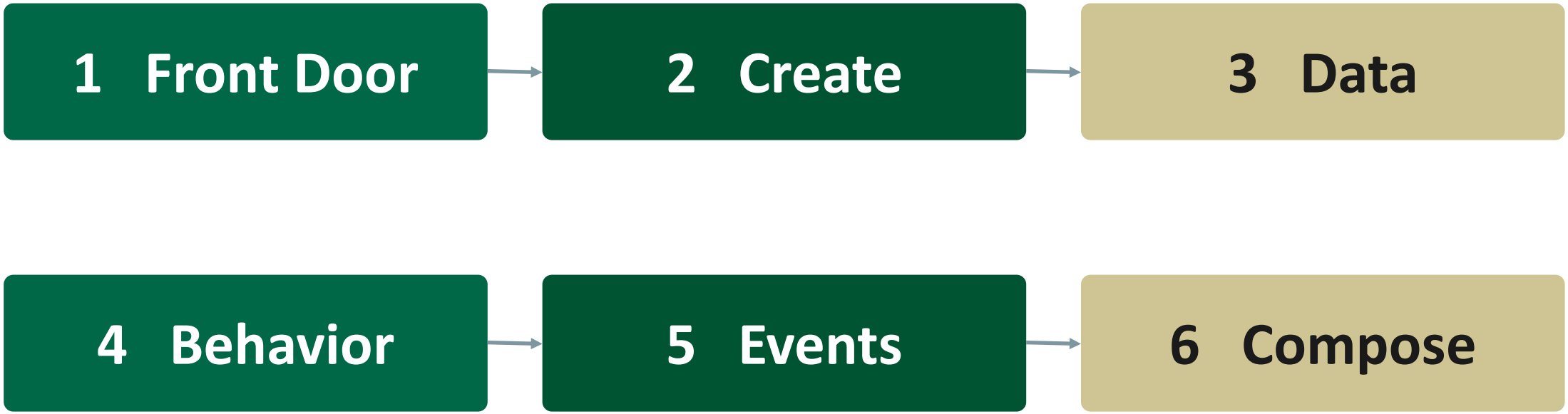
"SOLID" and Separation of Concerns

Letter	Principle	Where you have already seen it
S	**Single Responsibility:** one reason to change	Why we decompose modules the way we do
O	**Open, Closed:** open to extension, closed to modification	Strategy: add behavior without editing existing code
L	**Liskov Substitution:** a subclass must honor its parent's contract	Keeps Factory Method products interchangeable
I	**Interface Segregation:** many small interfaces beat one large one	Adapter narrows what a client actually depends on
D	**Dependency Inversion:** depend on an abstraction, not a concrete class	Repository and Model Gateway apply this directly

SOLID is the theory that explains why these patterns work.

One Request's Journey

Each stop below will be explained



Six stops. Six questions your own project will have to answer too.

Adapter and Model Gateway

- **Adapter:**
  - translates a legacy or third-party interface into one your client already expects.
- **Use when:**
  - an outside interface does not match yours.
- **AI angle:**
  - wrap AI generated calls to third party code behind one Adapter.

- **Model Gateway:**
  - the same idea, applied to AI calls, so providers can be swapped safely.
- **Use when:**
  - you need to swap AI models or add a fallback.
- **AI angle:**
  - every AI call in your Support Ticket System should pass through one Gateway.

Factory Method and Builder

- **Factory Method:**
  - creates a Ticket, EscalatedTicket, or BugTicket without knowing which in advance.
- **Use when:**
  - new related types should not touch existing code.
- **AI angle:**
  - verify an AI generated suggestion actually fits your product family.

- **Builder:**
  - assembles a Ticket's many optional parts in a specific order.
- **Use when:**
  - optional parts are added step by step.
- **AI angle:**
  - have AI build one step at a time, then review the result.

Singleton, and Why to Be Careful With It

- Guarantees exactly one instance of a class, with one global access point
- Tempting for configuration objects, loggers, or a database connection
- The catch: it makes unit testing harder, since state persists across tests
- The catch: a naive version can create two instances under concurrency
- Ask whether passing the object explicitly would work just as well
- AI angle: AI often defaults to Singleton for anything global; ask the question above first.

Repository

Gives the rest of the application one clean, collection like path to the data store.



Use when: you want to isolate data access so storage can change later. The next two slides build on this.

Retrieval Augmented Generation (RAG)

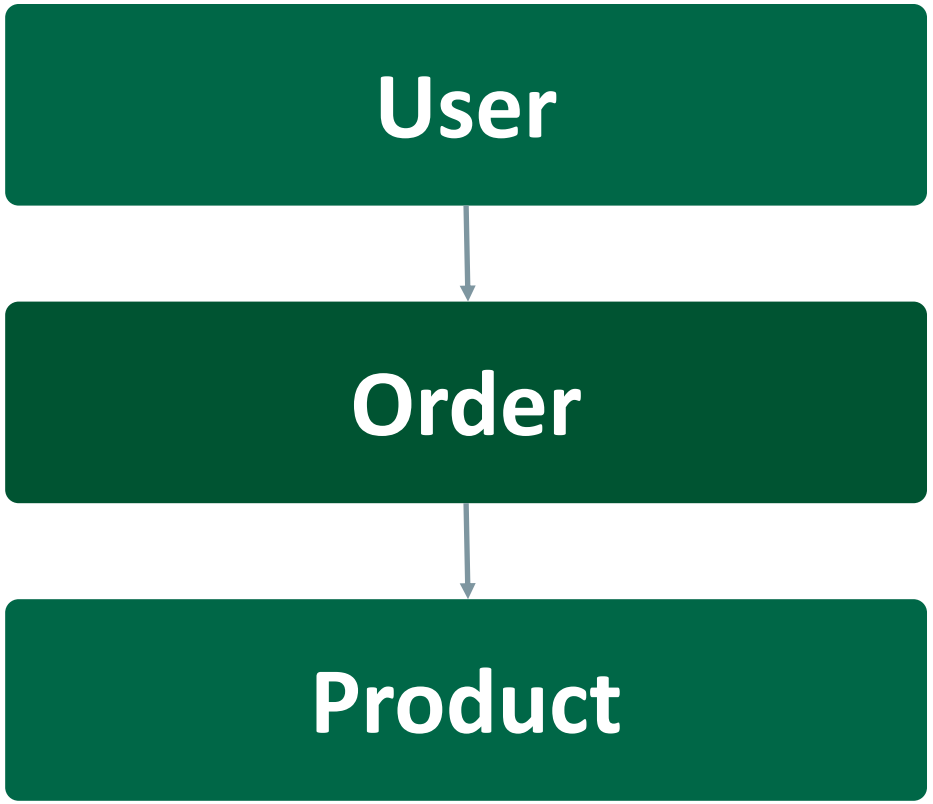
Repository gave one path to the data store. RAG grounds AI answers in that same data.



Use when: your AI feature must answer from your own data, not just the model's general knowledge.

Representing Your Database Schema

- Show entities, their key attributes, and relationships
- Conceptual level: no need for every column and constraint
- AI can draft a first schema fast; you verify it matches requirements
- Your Repository and RAG retriever both read from this schema.



Strategy and State

- **Strategy:**
  - makes an algorithm swappable, such as how SLA deadlines are calculated.
- **Use when:**
  - you expect to add or swap a rule without touching calling code.
- **AI angle:**
  - ask AI for one Strategy class per rule, not one function full of flags.

- **State:**
  - a Ticket behaves differently in Draft, Submitted, and Approved.
- **Use when:**
  - allowed operations depend on the object's current state.
- **AI angle:**
  - ask AI for one class per state, not one giant if or switch block.

Observer and Command

- **Observer:**
  - notifies every interested part of the system when a Ticket's state changes.
- **Use when:**
  - several parts must react to one event.
- **AI angle:**
  - an AI generated notification feature is usually just a new Observer.

- **Command:**
  - turns closing, reopening, or reassigning a Ticket into a loggable, undoable object.
- **Use when:**
  - you need undo or redo, a queue, or an audit log.
- **AI angle:**
  - AI generated automation scripts fit naturally as Command objects.

MVC and MVVM as Composite Patterns

- MVC, Model View Controller, and MVVM, Model View ViewModel, combine several patterns to separate concerns
- Model: data and rules. View: what the user sees. Controller or ViewModel: the glue
- Facade applies the same idea you saw at the front door: one simple entry point, now in front of an internal subsystem



Data flows both directions: user actions flow left, updated state flows right.

CAUTION

Anti Patterns to Avoid

God Object

One class or module that does almost everything. It becomes the bottleneck everyone must touch.

Golden Hammer

Applying your one favorite pattern to every problem, whether it fits or not.

Spaghetti Logic

Control flow so tangled that no one can trace it a week later.

The Finished Map

Here is what filled each stop below.

1 Front Door →	**2 Create** →	**3 Data**
Adapter, Gateway	Factory, Builder, Singleton	Repository, RAG, Schema
4 Behavior →	**5 Events** →	**6 Compose**
Strategy, State	Observer, Command	MVC, MVVM, Facade

Your own project MUST answer these same six questions.

Choosing Patterns: A Decision Guide

- **Swap a rule at runtime?**
  - Strategy
- **React to events across the system?**
  - Observer
- **Need controlled object creation?**
  - Factory Method or Builder
- **Need to swap AI models?**
  - Model Gateway
- **Wrap incompatible interfaces?**
  - Adapter
- **One entry to a complex subsystem?**
  - Facade
- **One clean path to your data store?**
  - Repository, then MVC or MVVM
- **Need answers grounded in your data?**
  - Retrieval Augmented Generation

AI Based Design Pattern details.

Bringing It to Your Project

- Front Door and Create map to your DDD's Design Patterns and Principles section.
- Data maps to your DDD's System Configuration Details and Architecture Decision Records sections.
- Swappable Behavior and Reacting and Recording map to your DDD's Module Specifications section.
- Holding It Together maps to your DDD's Architectural Views, Logical and Behavioral, sections.
- Every pattern you chose needs one sentence in your DDD explaining **why**, not just that you used it.

02

AI in Design and Architecture

Specific prompts, ADRs, and verification

Where AI Genuinely Helps

- Drafting a first pass HLA or component diagram from your requirements
- Suggesting design patterns that fit a described problem
- Writing first drafts of ADRs and DDD sections
- Generating a starting database schema from entity descriptions
- Scaffolding boilerplate code once architecture and patterns are agreed

Practical Tool Categories

Category	Job it does for your team
AI coding assistants	Generate and refactor code inside your editor
AI diagramming	Turn a written description into an architecture diagram
AI documentation	Draft ADRs, DDD sections, and code comments

Prompting AI for an HLA Draft

Example Prompt

```
Given these requirements, propose a component level
HLA. List each component, its one sentence
responsibility, and the data that flows between
components. Do not invent requirements I did not give
you. Ask any clarifying questions as needed.
```

- Named components tied directly to your requirements
- One responsibility sentence per component
- Explicit data flow between components

Prompting AI for Pattern Selection

Example Prompt

```
Here is a module's responsibility and here are my
constraints on team size, timeline, and expected
changes. Which pattern fits best, and which is the
runner up? State the tradeoff of each in two
sentences. Be concise and ask questions if needed.
```

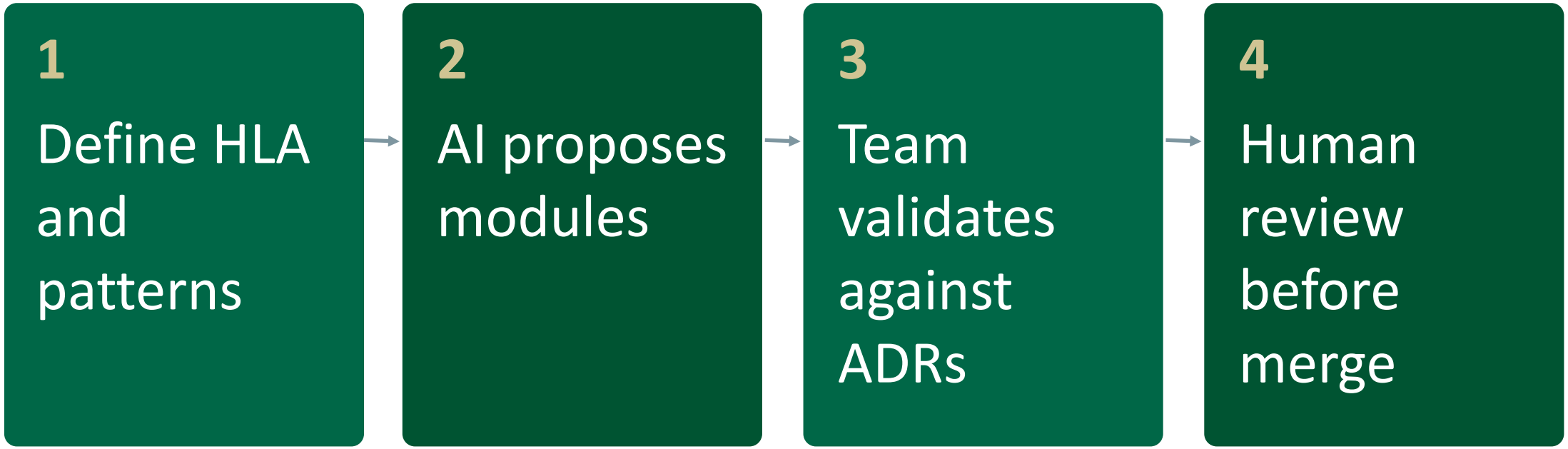
- Does the reasoning match your real constraints, not generic advice?
- Did it name a real tradeoff, not just praise its own pick?

Writing ADRs With AI Assistance

Architecture Decision Records *(ADRs) answer the "Why" of a specific design choice. This is a huge portion of the history & versioning you must track at all time!*

Field	Filled example
Title	ADR 007: Repository for order data
Decision	Introduce OrderRepository; no direct queries
Consequences	One extra layer; storage can change freely later

An AI Assisted Design Workflow



*The human validation step **never** disappears; now it checks both requirements and ADRs.*

Verifying AI Output Against Specifications

- Does this satisfy the requirement it claims to, with nothing invented?
- Does it use the pattern your team already agreed on?
- Does it introduce a dependency nobody has approved?
- Is it consistent with the coupling and cohesion choices from Week 6?

Sample Prompt to alternate AI Model (AI Checking AI)

```
Given the coupling, cohesion model & Design pattern used
and agreed upon, indicate and show gaps with the current
design for this specific specification.
```

03

AI Pitfalls and Guardrails

Six specific ways this goes wrong

Three (AI) Pitfalls to Watch For

Hallucinated Architecture

AI invents components, dependencies, or APIs that do not actually exist. Occurs in more complex and long sessions.

Inconsistent Decisions

Each teammate prompts AI separately and gets an uncoordinated design. Usually caused by poor input prompts.

Data Exposure

Pasting proprietary requirements or code into a public tool without permission. Keeps Security teams up at night answering questions from Legal, Compliance & CIO!

Three More (AI) Pitfalls to Watch For

Shallow Understanding

A team ships code nobody on the team can actually explain or defend. What type of testing was done? (WAY TO COMMON)

Pattern Misapplication

AI suggests a pattern that fits the prompt's words but not the real problem. (VERY COMMON)

Dependency Drift

AI recommends a library or version that conflicts with what the team already chose. Occurs in systems with many app versions.

Guardrails That Keep AI Honest

- Write an ADR before any AI proposed architecture change gets merged and have human review.
- One teammate owns the master HLA; every AI suggestion is checked against it by another team-mate.
- Agree on one team prompt template for design questions or develop a specific “Skill” (preferred).
- Never paste client data or proprietary specs into a public AI tool.
- Every AI generated component gets a human code review before merging.

04

Methodology for Rapid, AI Assisted Builds

Agile, AI SE, TDD, and FDD in fifteen weeks

Four Methodologies at a Glance

Agile

Short cycles, frequent feedback, adapting the plan as you learn

AI SE

Software engineering practice that treats AI as an active team member

TDD

Write the test before the code it is meant to verify

FDD

Organize work around a list of small, client valued features

Deliveries Mapped to a Fifteen-Week Timeline

Weeks	Phase focus
1 to 5	Requirements, Specifications, HLA, and initial design, start scaffolding. Identify Edge cases.
6 to 8	Detailed design, patterns, and setup. Complete core test concepts, system setup.
9 to 12	Build in short 1 week sprints with AI assistance.
13 to 15	Integration, User Testing, and Final Delivery.

Each phase spans several one-week sprints, as defined in Week 6. (Agile or Waterfall?)

TDD as the Safety Net for AI Generated Code

- Write the test first, from your design, before AI generates the code
- A failing test tells you immediately when generated code drifts
- Tests are the clearest way to check AI output against requirements
- This is how a small team moves quickly without moving blindly

FDD for Small Team Feature Decomposition

- Build a short list of concrete, client valued features
- Each feature is small enough for one person to own start to finish
- Features map naturally to your architecture's modules from Week 6
- Three or four owned feature lists beat one tangled shared backlog

Recommended Hybrid Approach for This Course

Agile

Weekly check ins, adjust the plan as the build teaches you something. Meet as a full team at least twice a week, three to four times preferred.

FDD

Split the backlog into owned features across the team. Each person must produce verifiable / testable features.

TDD

Every feature gets a test before AI or a teammate writes the code.

AI SE

AI drafts, a human reviews, the ADR records the decision, following your project sponsor's direction on how much AI assistance is expected.

Key Takeaways



- Choose Design patterns from the problem you have, using the decision guide. Ensure you can explain the Coupling & Cohesion used in your projects!
- A good prompt names your constraints; a good ADR records the decision.
- Six pitfalls are named. Check for them every sprint, every application decision, not just once.
- Agile, FDD, TDD, and AI SE combine into one hybrid rhythm for the build in this class. Understand Agile & Waterfall and how & when they are used.

Questions?

CIS 4910, Week 7

Professor Scot Hollingsworth

scoth@usf.edu

